F5 AI Security Guardrails on Amazon EKS

This repository automates a single-node Amazon EKS proof of concept for F5 AI Security Guardrails.

The PoC uses:

- Amazon EKS with one GPU managed node group
- F5 AI Security Operator installed from Harbor
- Guardrails inference enabled
- Red Team disabled
- In-cluster PostgreSQL
- NGINX Ingress Controller
- Route 53 DNS
- Self-signed TLS for lab access

> This is a PoC, not a production reference architecture. Do not commit real Harbor credentials, license strings, passwords, or generated secrets.

Architecture

```
Browser
  |
  | https://guardrails.f5demo.io
  v
Route 53 CNAME
  |
  v
AWS Network Load Balancer
  |
  v
NGINX Ingress Controller
  |
  +-- /auth -> cai-moderator:8080  (Keycloak)
  |
  +-- /     -> cai-moderator:5500  (F5 AI Security UI/API)
```

Application Namespaces

Namespace	Purpose
`f5-ai-sec`	F5 AI Security Operator
`cai-moderator`	Guardrails UI/API, Keycloak auth, PostgreSQL
`prefect`	Prefect server, worker, workflow registration
`f5-ai-sec-inference`	Guardrails inference service and scanner model
`ingress-nginx`	Public ingress controller and AWS load balancer

Sizing Note

The F5 runbook recommends more headroom for single-node deployments. This PoC uses `g5.4xlarge` as the smallest practical AWS GPU instance because it provides:

- 1 NVIDIA A10G GPU with 24 GiB VRAM

- 16 vCPU
- 64 GiB memory

If pods remain Pending, are OOMKilled, or model loading becomes unstable, resize to `g5.8xlarge`.

Prerequisites and Preflight

The script checks prerequisites before deployment. You do not need to manually run every AWS and Kubernetes verification command unless troubleshooting.

Required local tools:

- `aws`
- `kubectl`
- `helm`
- `eksctl`
- `openssl`
- `python3`
- `curl`

Required inputs:

- AWS CLI credentials with permissions for EKS, EC2, IAM, CloudFormation, ELB, EBS, and Route 53
- Route 53 hosted zone for the selected hostname
- Harbor registry credentials for `harbor.calypsoai.app`
- F5 AI Security license string
- LLM provider credentials for post-install app configuration

Configure AWS on each machine before running the script:

```
aws configure
# or
export AWS_PROFILE=<profile-name>
aws sso login --profile <profile-name>
```

Run preflight:

```
./scripts/guardrails-poc.sh preflight
```

The preflight check validates local tools, AWS identity, region, Route 53, GPU instance availability, GPU quota visibility, Harbor login, license input, and `eksctl` cluster configuration.

Script Deployment

Use the script for normal PoC lifecycle operations.

```
./scripts/guardrails-poc.sh preflight
./scripts/guardrails-poc.sh up
./scripts/guardrails-poc.sh status
./scripts/guardrails-poc.sh down --yes
```

The script is idempotent. If deployment stops partway through, rerun `up`; it reuses existing AWS and Kubernetes resources and continues from the current state.

Configure Inputs

Copy the example config:

```
cp config/guardrails-poc.env.example config/guardrails-poc.env
```

Edit `config/guardrails-poc.env` if you need to change region, cluster name, hostname, hosted zone, instance type, or secret file paths.

Default secret paths:

```
HARBOR_CREDENTIALS_FILE="config/harbor.txt"
F5_LICENSE_FILE="config/license.txt"
```

Relative paths are resolved from the repository root.

Create local secret files:

```
printf '%s\n%s\n' '<harbor username>' '<harbor password>' > config/harbor.txt
printf '%s\n' '<F5 license string>' > config/license.txt
chmod 600 config/harbor.txt config/license.txt
```

These files are ignored by Git. You can also supply secrets with environment variables:

```
export HARBOR_USERNAME='<harbor username>'
export HARBOR_PASSWORD='<harbor password>'
export F5_LICENSE_STRING='<F5 license string>'
```

Deploy

```
./scripts/guardrails-poc.sh up
```

Expected runtime is roughly 25-45 minutes for a new environment because EKS control plane, GPU node, add-ons, images, model pod, ingress load balancer, and DNS all need time to come online.

At the end, the script prints a concise status summary with component health, UI URL, initial login credentials, and password-change links.

Status

```
./scripts/guardrails-poc.sh status
```

Example:

```
Guardrails PoC status
=====================
Environment:               running
Public URL:                online (https://guardrails.f5demo.io)
DNS:                       present -> <load-balancer>
Ingress LB:                ready -> <load-balancer>
EKS cluster:               ACTIVE, Kubernetes 1.35, region us-east-1
Node:                      1/1 Ready, type g5.4xlarge
Storage:                   ready (EBS CSI active, gp2 default)
f5-ai-security-operator:   ready (1/1 replicas)
cai-moderator:             ready (2/2 pods)
f5-ai-sec-inference:       ready (2/2 pods)
prefect:                   ready (3/3 pods)
ingress-nginx:             ready (1/1 replicas)
EBS leftovers:             none found
```

Stop and Remove Cost

```
./scripts/guardrails-poc.sh down --yes
```

This deletes Route 53 records, the EKS cluster, GPU node group, load balancer, EBS volumes created through the cluster, available leftover cluster-owned EBS volumes, and `eksctl` CloudFormation stacks.

This delete-and-recreate model is slower than scaling a node group to zero, but it removes EKS control-plane and GPU-node cost.

First Login and Password Change

Initial credentials from the F5 runbook:

```
Username: admin
Password: pass
```

The Guardrails UI account page does not expose password management. Use the embedded Keycloak account console.

For this PoC, change the password in the `master` realm:

```
https://guardrails.f5demo.io/auth/realms/master/account/#/security/signingin
```

The deployment can also expose a `calypsoai` realm:

```
https://guardrails.f5demo.io/auth/realms/calypsoai/account/#/security/signingin
```

Keycloak passwords are realm-specific. If more than one realm accepts the initial password, rotate it in each realm.

Manual Deployment Reference

The script is the recommended path. Use this section only if you want to understand or manually reproduce what the script does.

1. Create EKS

```
eksctl create cluster \
  --name f5-guardrails-poc \
  --region us-east-1 \
  --version 1.35 \
  --managed \
  --nodegroup-name guardrails-gpu-ng \
  --node-type g5.4xlarge \
  --nodes 1 \
  --nodes-min 1 \
  --nodes-max 1 \
  --node-volume-size 150 \
  --node-ami-family AmazonLinux2023 \
  --install-nvidia-plugin \
  --vpc-nat-mode Disable
```

2. Prepare Storage

```
eksctl create addon \
  --cluster f5-guardrails-poc \
  --region us-east-1 \
  --name eks-pod-identity-agent \
  --force

eksctl create addon \
  --cluster f5-guardrails-poc \
  --region us-east-1 \
  --name aws-ebs-csi-driver \
  --auto-apply-pod-identity-associations \
  --force

kubectl patch storageclass gp2 \
  -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

3. Install Operator and Guardrails

```
kubectl create namespace f5-ai-sec

kubectl create secret docker-registry regcred \
  --docker-server='harbor.calypsoai.app' \
  --docker-username='<HARBOR_USERNAME>' \
  --docker-password='<HARBOR_PASSWORD>' \
  -n f5-ai-sec

helm registry login harbor.calypsoai.app

helm upgrade --install f5-ai-security-operator \
  oci://harbor.calypsoai.app/calypsoai/f5-ai-security-operator-helm \
  --version 1.4.1 \
  -n f5-ai-sec
```

Apply a Guardrails-only `SecurityOperator` custom resource with:

- `registryAuth.existingSecret: regcred`
- in-cluster PostgreSQL enabled
- `CAI_MODERATOR_BASE_URL` set to `https://guardrails.f5demo.io`
- `CAI_MODERATOR_DEFAULT_LICENSE` set to your F5 license string
- Guardrails inference enabled
- Red Team disabled

4. Complete Supporting Services

The first manual deployment required these fixes, which are now automated by the script:

- Wait for Prefect API health before recreating the workflow registration job.
- Install ingress-nginx with an internet-facing AWS NLB.
- Create a self-signed TLS secret for `guardrails.f5demo.io`.
- Create an ingress routing `/auth` to port `8080` and `/` to port `5500` on `cai-moderator`.
- Create a Route 53 CNAME from `guardrails.f5demo.io` to the ingress-nginx load balancer.

Troubleshooting

Use the script summary first:

```
./scripts/guardrails-poc.sh status
```

Useful detailed commands:

```
kubectl get pods -A
kubectl get svc -n ingress-nginx ingress-nginx-controller -o wide
kubectl get ingress -n cai-moderator -o wide
kubectl logs -n cai-moderator deploy/cai-moderator --tail=100
kubectl logs -n f5-ai-sec deploy/controller-manager --tail=100
kubectl logs -n f5-ai-sec-inference deploy/f5-ai-sec-inference --tail=100
kubectl logs -n prefect deploy/prefect-server --tail=100
```

If the public URL is not reachable immediately after Route 53 is updated, wait a few minutes for DNS propagation and local resolver cache refresh.

Production Considerations

For production, change the following:

- Use trusted TLS with ACM or cert-manager.
- Use external PostgreSQL such as Amazon RDS.
- Use private node networking and controlled egress.
- Add backup and restore for PostgreSQL.
- Use larger or separate node groups for CPU and GPU workloads.
- Use least-privilege AWS IAM permissions.
- Store secrets in a proper secret manager.